

Polimorfismo

Introduzione

In programmazione ad oggetti ([OOP](#)), il **polimorfismo** significa: *"avere molte forme"*.

Concretamente il polimorfismo **permette di definire metodi con lo stesso nome in classi diverse, ma ognuno con un comportamento specifico**.

Questo è finalizzato in Python a rendere il codice più **flessibile, riutilizzabile e manutenibile**.

Cos'è il polimorfismo

In sostanza il polimorfismo è:

la possibilità di invocare lo stesso nome di un metodo su oggetti (istanze) di classi diverse, ottenendo risultati diversi a seconda dell'oggetto.

^polimorfismo-def

Il suo obiettivo è rendere il codice capace di gestire oggetti di tipo diverso usando una interfaccia comune (cioè usando uno stesso metodo con lo stesso nome).

In Python, **non serve** dichiarare esplicitamente il tipo degli oggetti (come in Java o C++), quindi il polimorfismo avviene in modo naturale: si parla infatti di **polimorfismo duck typing** ("se cammina come un'anatra e starnazza come un'anatra, allora è un'anatra").

Esempio pratico di Polimorfismo

Step 1: File `Persona_2.py`

1. Andiamo a creare un file `.py` chiamato `Persona_2.py`, in questo file creiamo una classe `Persona`:



Python

```
1 class Persona:
```

2. dopodiché dobbiamo definire il **costruttore della classe** (`__init__()`), che prende:

come parametri l'attributo `name` (cioè il nome della persona), l'attributo `last` (cioè il lastname o cognome in italiano) e, infine, l'attributo `age` (cioè l'età della persona).



Python

```
1 class Persona:
2
3     def __init__(self, name:str, last:str, age:int):
4
5         self.name = name
6
7         self.lastname = last
8
9         self.age = age
```

3. Inoltre implemento anche una serie di setter e getter all'interno della classe e il metodo speciale `__str__()`:



Python

```
1     '''
2
3     Mi consente di impostare un valore per self.name
4
5     '''
6
7     def set_name(self, name: str) -> None:
8
9         self.name = name
10
11     def __str__(self):
12
13         return f"{self.name} {self.lastname}, {self.age} anni"
14
15     '''
16
17     Mi consente di impostare un valore per self.lastname
18
19     '''
20
21     def set_lastname(self, lastname: str) -> None:
22
23         self.lastname = lastname
24
25     '''
26
27     Mi consente di impostare un valore per self.age
```

```
28
29     '''
30
31     def set_age(self, age: int) -> None:
32
33         if age < 0 or age > 130:
34
35             self.age = 0
36
37         else:
38
39             self.age = age
40
41     '''
42
43     funzione che mi consente di ritornare il valore di self.Name
44
45     '''
46
47     def getName(self) -> str:
48
49         return self.name
50
51     '''
52
53     funzione che mi consente di ritornare il valore di
self.lastname
54
55     '''
56
57     def getLastName(self) -> str:
58
59         return self.lastname
60
61     '''
62
63     funzione che mi consente di ritornare il valore di self.age
64
65     '''
66
67     def getAge(self) -> int:
68
69         return self.age
70     def display_data(self) -> None:
```

```

71
72         print(f"Nome: {self.name}")
73
74         print(f"Cognome: {self.lastname}")
75
76         print(f"Età: {self.age}")

```

Da notare come il metodo `set_age()` lo si usi anche per controllare se il valore dell'attributo `age` sia un valore valido:

all'interno del metodo `def set_age(self):` impostiamo un blocco `if-else` per cui:

se il valore dell'attributo `age` è minore di `0` o maggiore di `130` allora `self.age = 0`,
altrimenti (`else`); `self.age = age`.

4. Infine come ultima cosa, ma non per importanza, definiamo un metodo `speak()`, che andrà a stampare in output un messaggio di saluto:



Python

```

1     def speak(self) -> None:
2
3         print(f"Hello! My name is {self.getName()}")

```

Step 2: File `alieno.py`

1. Creiamo un file chiamato `alieno.py` e all'interno definiamo una classe `Alieno`:



Python

```

1     class Alieno:

```

2. Di un alieno ci interessa sapere la galassia di provenienza, per questo con il costruttore della classe andremo a definire un parametro `galaxy:str`:



Python

```

1     class Alieno:
2         def __init__(self, galaxy:str):
3             self.setGalaxy(galaxy)

```

Più o meno come per l'attributo `age` , il `self.setGalaxy(galaxy)` serve per controllare che il valore di `galaxy` sia valido, ad esempio che non sia una stringa vuota.

3. Andiamo a definire un metodo per impostare il valore dell'attributo `self.galaxy` :



Python

```
1     def setGalaxy(self, galaxy:str)->None:
2
3         if galaxy:
4
5             self.galaxy=galaxy
6
7         else:
8
9             print("Errore! La galassia non può essere una stringa
              vuota")
```

In questo metodo settiamo, attraverso l'uso di un setter, il valore di `galaxy` dentro un blocco `if-else` :

nello statement `if` , andiamo a dire che se `galaxy` è `True` (cioè in questo caso non è una stringa vuota) allora deve impostare `self.galaxy = galaxy` , altrimenti deve stampare in output un messaggio di errore dicendo che la galassia non può corrispondere a una stringa vuota.

4. In seguito definiamo un metodo per restituire il valore dell'attributo `self.galaxy` :



Python

```
1     def getGalaxy(self) -> str:
2         return self.galaxy
```

5. Come per la classe `Persona` , definiamo un metodo `speak()` , che stampa in output un messaggio di saluto:



Python

```

1     def speak(self)->None:
2
3         print("hfdhfdjhfhjdffidshfdjkh")

```

- Andiamo a definire il metodo speciale `__str__()` che ritorna in output la una stringa formatta:

```

Python
1     def __str__(self)->str:
2         return f"Alieno proveniente dalla galassia
        {self.getGalaxy()}"

```

Step 3: File `polimorfismo.py`

- Andiamo a creare il file `polimorfismo.py` e importiamo sia dal file `Persona_2.py` la classe `Persona` che dal file `alieno.py` la classe `Alieno`

```

Python
1  from Persona_2 import Persona
2  from alieno import Alieno

```

- In seguito creiamo un oggetto o istanza di `Persona` (cioè l'istanza `p`) e stampiamo in output le informazioni dell'oggetto `p` della classe `Persona`:

```

Python
1  p:Persona = Persona("Antonio", "Scugnizzo", 22)
2  # Visualizzare le informazioni dell'oggetto p della classe
    Persona
3  print(p) #Output: Antonio Scugnizzo, 22 anni

```

- Dopodiché creiamo l'istanza di `Alieno` (cioè l'istanza `et`) e stampiamo le informazioni dell'oggetto `et` della classe `Alieno`:

```

Python
1  et:Alieno = Alieno("Andromeda")
2  # Visualizzare le informazioni dell'oggetto et della classe
    Alieno
3  print("\n",et)

```

4. Adesso possiamo invocare il metodo `speak()` sia della classe `Persona` che della classe `Alieno` :



Python

```
1  '''
2  invocare il metodo speak() della classe Persona
3  '''
4  p.speak()
5
6  '''
7  invocare il metodo speak() di un ALieno
8  '''
9  et.speak()
```

Se runniamo il codice vediamo che l'output corrisponderà sia al messaggio del `print()` contenuto nella funzione `speak()` della classe `Persona` , che al messaggio del `print()` contenuto nella funzione `speak()` della Classe `Alieno` . Quindi in output abbiamo 2 risultati diversi: prima diciamo all'oggetto `p` invoca il metodo `speak()` che compie un'azione specifica relativa alla classe `Persona` , mentre l'oggetto `et` invoca il metodo `speak()` che mi fa una azione specifica per la classe `Alieno` .

In sintesi, come accennato prima: **il polimorfismo è quindi avere metodi con lo stesso nome per classi diverse e definiti in maniera diversa per ogni classe.**

Differenza tra Overriding e Polimorfismo

Sebbene l'overriding e il polimorfismo di primo impatto possano sembrare due concetti simili, non lo sono.

Semmai sono due concetti **correlati ma distinti**.

Overriding

L'**overriding** avviene quando una sottoclasse ridefinisce un metodo ereditato da una superclasse.

Serve per modificare o personalizzare il comportamento di un metodo già definito nella classe genitore.



Python

```
1  class Alieno:
```

```

2     def speak(self):
3         print("Lingua aliena: bzz bzz bzz")
4
5     class Persona(Alieno): # Persona eredita da Alieno
6         def speak(self): # Override del metodo speak
7             print("Ciao, sono una persona")
8
9     p = Persona()
10    p.speak() # --> "Ciao, sono una persona"
11
12    a = Alieno()
13    a.speak() # --> "Lingua aliena: bzz bzz bzz"

```

Spiegazione:

- Anche se entrambi gli oggetti (`p` e `a`) hanno un metodo `speak()` , quello della **sottoclasse** (`Persona`) **ha sovrascritto** il comportamento del genitore.
- Python guarda il **tipo dell'oggetto** (`Persona` o `Alieno`) e usa il metodo corrispondente.

Cos'è invece il Polimorfismo?

Il **polimorfismo** è un concetto più generale:

Permette di usare lo stesso nome di metodo su oggetti di classi diverse, indipendentemente dal fatto che una classe erediti dall'altra.

Esempio:



Python

```

1     class Persona:
2         def speak(self):
3             print("Ciao! Sono umano.")
4
5     class Alieno:
6         def speak(self):
7             print("Krrgztt! Sono alieno.")
8     def chi_parla(essere):
9         essere.speak()
10
11    chi_parla(Persona()) # --> "Ciao! Sono umano."
12    chi_parla(Alieno()) # --> "Krrgztt! Sono alieno."

```

Qui, non c'è **nessun legame di ereditarietà**, ma Python può comunque chiamare `speak()` su entrambi gli oggetti.

Questo è **polimorfismo tramite duck typing**: se un oggetto ha il metodo `speak()`, allora può parlare, **a prescindere da che classe appartenga**.

Quindi come si fa a distinguere i due metodi delle due classi diverse?

Dipende dall'oggetto che lo invoca:

se l'oggetto `p` invoca il metodo `speak()` allora sto dicendo a Python di invocare il metodo `speak()` della classe `Persona`, se l'oggetto `et` invoca il metodo `speak()` sto dicendo a Python di invocare il metodo `speak()` della classe `Alieno`.

Python non guarda il nome del metodo, ma a quale classe appartiene l'oggetto che lo invoca.